Nome: Lucas Lima de Barros Lica. **Turma:** 2°AMS.

Laboratório 02 — Padrão de Projeto Factory Method

Atividade 1 — Analisando o problema

3.1 — Quais classes concretas SistemaNotificacao conhece?

R: A classe SistemaNotificacao conhece diretamente três classes concretas: EmailNotificacao, SMSNotificacao e WhatsAppNotificacao. Isso pode ser observado porque o próprio código possui referências explícitas a essas classes e utiliza o operador new para instanciá-las. Essa situação é importante porque SistemaNotificacao passa a depender dos detalhes das implementações concretas. Em outras palavras, ela não conhece apenas a ideia geral de uma notificação; ela precisa saber exatamente quais classes existem e como instanciá-las. Esse conhecimento direto aumenta o acoplamento, pois uma alteração na estrutura de criação das notificações pode exigir alterações em SistemaNotificacao.

3.2 — Onde os objetos são criados?

R: Os objetos são criados dentro do método enviar() da classe SistemaNotificacao. No código apresentado, cada bloco condicional cria diretamente uma implementação concreta. Por exemplo, quando o tipo é EMAIL, o programa executa new EmailNotificacao(), quando é SMS, executa new SMSNotificacao(), e quando é WHATSAPP, executa new WhatsAppNotificacao(). Isso significa que a mesma classe concentra três responsabilidades: decidir qual tipo utilizar, criar o objeto e executar a operação de envio. Essa concentração é justamente o problema que o Factory Method pretende reduzir.

3.3 — O que acontecerá se a empresa criar uma nova forma de notificação?

R: Será necessário acrescentar uma nova condição no código para reconhecer o novo tipo e, dentro dessa condição, instanciar a nova classe. Por exemplo, se fosse criada uma notificação Push, seria necessário acrescentar uma condição semelhante a if (tipo.equals("PUSH")) e executar new PushNotificacao(). O problema é que o crescimento da quantidade de canais faz o método enviar() ficar cada vez maior e mais dependente de classes concretas. Portanto, adicionar funcionalidades passa a exigir alterações em uma classe que já possui a responsabilidade de utilizar as notificações.

3.4 — Seria necessário modificar SistemaNotificacao?

R: Sim. Na solução inicial, seria necessário modificar SistemaNotificacao sempre que um novo canal fosse criado. A razão é que a própria classe possui as condições que determinam qual objeto será instanciado. Assim, a criação de Push, Telegram ou qualquer outro canal exigiria acrescentar novas condições e novos comandos new. Esse comportamento é indesejável porque faz com que uma classe existente seja constantemente modificada à medida que o sistema cresce.

3.5 — Qual parte do código poderia ser responsável pela criação dos objetos?

R: A responsabilidade poderia ser transferida para uma classe especializada em criação, representada no laboratório por NotificacaoFactory e pelas suas subclasses. A fábrica abstrata define o método criarNotificacao(), enquanto cada fábrica concreta decide qual objeto específico deve ser criado. Dessa maneira, a classe que utiliza

a notificação deixa de precisar conhecer todos os comandos new. A criação fica encapsulada em um ponto apropriado do projeto, reduzindo o acoplamento entre o código cliente e os produtos concretos.

Atividade 2 — Modelagem inicial

4 — Construa um Diagrama de Classes UML para a solução inicial.

R: A solução inicial deve representar SistemaNotificacao como a classe que possui o método enviar() e que depende diretamente das classes EmailNotificacao, SMSNotificacao e WhatsAppNotificacao. Cada uma das classes de notificação deve possuir o método enviar(String mensagem). A característica mais importante do diagrama é mostrar que SistemaNotificacao está ligada diretamente às implementações concretas. Essa representação evidencia visualmente o problema apresentado no laboratório: a classe responsável pelo funcionamento do sistema também está responsável pela escolha e criação dos objetos. Portanto, o diagrama inicial serve como comparação com o modelo posterior baseado em abstração e fábricas.

Atividade 3 — Criando o produto

7 — Qual é a função da interface Notificacao?

R: A interface Notificacao representa o produto abstrato do padrão Factory Method neste laboratório. Ela estabelece um contrato comum para todos os canais de notificação. O contrato define o método enviar(String mensagem), indicando que qualquer classe que represente uma notificação deve ser capaz de enviar uma mensagem. A vantagem é que o restante do sistema pode trabalhar com Notificacao sem precisar saber se o objeto concreto é um e-mail, SMS, WhatsApp, Push ou Telegram. Isso cria uma abstração comum e permite que diferentes produtos sejam tratados de maneira polimórfica.

Atividade 4 — Criando os produtos concretos

8 — Quais são os produtos concretos?

R: Os produtos concretos inicialmente são EmailNotificacao, SMSNotificacao e WhatsAppNotificacao. Todos implementam a interface Notificacao, mas cada um representa um canal específico. Isso significa que todos possuem o mesmo contrato, representado por enviar(String mensagem), porém podem fornecer uma implementação própria para esse método. Essa estrutura é importante para o Factory Method porque as fábricas concretas posteriormente criarão esses produtos específicos.

Atividade 5 — Criando o Factory Method

9 — Qual é o Factory Method e qual é sua função?

R: O Factory Method é o método criarNotificacao(). Ele é declarado na classe abstrata NotificacaoFactory e possui como retorno o tipo abstrato Notificacao. Sua função é definir um ponto específico no qual a criação do produto acontece, sem obrigar a fábrica abstrata a determinar qual classe concreta será instanciada. A decisão é delegada

às subclasses, como EmailFactory e SMSFactory. Isso é fundamental porque permite que a classe abstrata trabalhe com uma abstração enquanto cada fábrica concreta fornece a implementação adequada da criação.

9.1 — Qual é a função de enviarMensagem()?

R: O método enviarMensagem() representa a utilização do produto criado. Primeiro, ele chama criarNotificacao() para obter uma Notificacao. Depois, chama enviar(mensagem) sobre o objeto retornado. A importância dessa separação é que enviarMensagem() não precisa saber qual classe concreta foi criada. Ele simplesmente sabe que recebeu um objeto que respeita o contrato Notificacao. Assim, a criação e a utilização ficam separadas, permitindo que diferentes fábricas produzam diferentes notificações sem alterar a lógica de envio da fábrica abstrata.

Atividade 6 — Criando as fábricas concretas

10 — Qual é a responsabilidade das fábricas concretas?

R: Cada fábrica concreta tem a responsabilidade de criar um produto específico. EmailFactory cria EmailNotificacao, SMSFactory cria SMSNotificacao, e WhatsAppFactory cria WhatsAppNotificacao. A justificativa para essa divisão é que a fábrica concreta é o local apropriado para conhecer a classe concreta que precisa ser instanciada. O restante da aplicação pode depender de NotificacaoFactory e Notificacao, sem precisar espalhar comandos new pelo código. Essa organização torna a inclusão de novos produtos mais previsível: para um novo canal, cria-se o produto e sua respectiva fábrica.

Atividade 7 — Utilizando a fábrica

11 — Qual é o resultado esperado e o que ele demonstra?

R: O programa deve apresentar uma mensagem indicando o envio por E-mail, outra indicando o envio por SMS e outra indicando o envio por WhatsApp. Mais importante do que a saída em si é observar que a mesma referência do tipo NotificacaoFactory pode receber diferentes fábricas concretas. Quando a referência aponta para EmailFactory, é criada uma EmailNotificacao, quando aponta para SMSFactory, é criada uma SMSNotificacao, e quando aponta para WhatsAppFactory, é criada uma WhatsAppNotificacao. Isso demonstra o uso de herança e polimorfismo na estrutura do Factory Method.

Atividade 8 — Entendendo o fluxo

12.1 — Quem solicita a criação?

R: O Main inicia o processo ao utilizar uma fábrica concreta e chamar enviarMensagem(). Dentro de enviarMensagem(), a própria NotificacaoFactory solicita a criação ao chamar criarNotificacao(). Portanto, o Main inicia a operação, mas a solicitação formal da criação acontece no fluxo interno da fábrica.

12.2 — Quem decide qual objeto será criado?

R: A fábrica concreta decide. Se a referência estiver apontando para EmailFactory, o método sobrescrito criarNotificacao() retorna EmailNotificacao. Se estiver apontando para SMSFactory, retorna SMSNotificacao, e

assim por diante. Essa decisão é justamente o papel do Factory Method: permitir que a implementação concreta determine qual produto deve ser produzido.

12.3 — Qual objeto efetivamente é criado?
R: No exemplo em que factory recebe uma instância de EmailFactory, o objeto efetivamente criado é EmailNotificacao. Embora a variável utilizada seja do tipo Notificacao, o objeto real possui a classe concreta EmailNotificacao. Isso demonstra polimorfismo: uma referência de uma abstração pode apontar para uma implementação concreta.

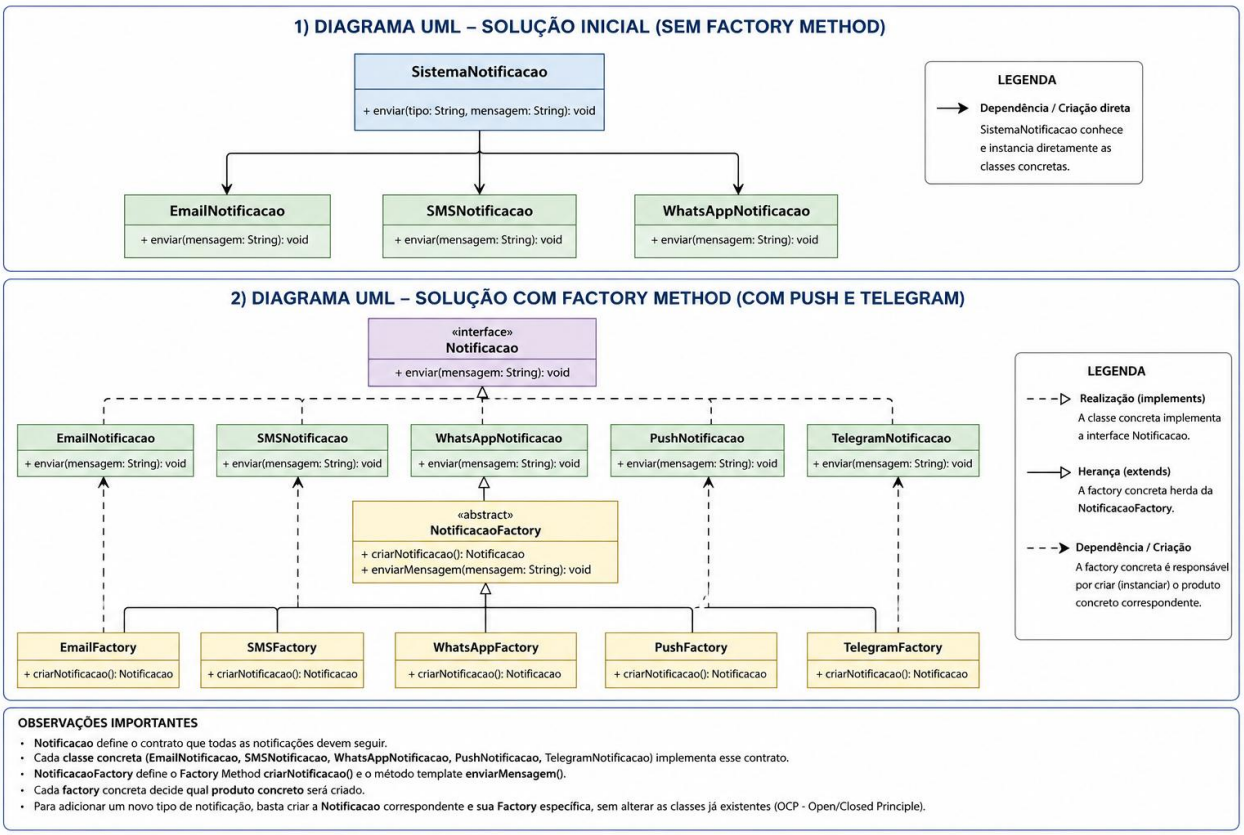
12.4 — Qual interface é utilizada pelo restante do sistema?
R: A interface utilizada é Notificacao. Ela funciona como o contrato comum entre os diferentes canais. O restante do sistema pode chamar enviar() sem precisar saber qual canal está por trás do objeto. Isso reduz a dependência de implementações concretas e facilita a substituição ou inclusão de novos produtos.

Atividade 9 — Novo tipo de notificação

13 — Como adicionar PushNotificacao?
R: Devemos criar PushNotificacao implementando Notificacao e PushFactory herdando de NotificacaoFactory. PushNotificacao deve implementar enviar(String mensagem), apresentando a saída indicada no laboratório. PushFactory deve implementar criarNotificacao() e retornar new PushNotificacao(). O ponto central é que não devemos alterar Notificacao, NotificacaoFactory, EmailFactory, SMSFactory ou WhatsAppFactory. Isso demonstra que a nova funcionalidade pode ser adicionada por extensão: criamos novos elementos compatíveis com as abstrações existentes, em vez de alterar todos os componentes anteriores.

Atividade 10 — Diagrama de Classes completo

14 — Como fica o Diagrama de Classes completo?
R: O modelo completo deve apresentar duas estruturas relacionadas. Na primeira, Notificacao é a interface e EmailNotificacao, SMSNotificacao, WhatsAppNotificacao e PushNotificacao são suas implementações concretas. Na segunda, NotificacaoFactory é a classe abstrata e EmailFactory, SMSFactory, WhatsAppFactory e PushFactory são suas subclasses. Cada fábrica concreta cria o produto correspondente. Essa relação entre duas hierarquias é importante: uma hierarquia representa os produtos que podem ser utilizados e a outra representa as fábricas responsáveis por produzi-los.



Atividade 11 — Desafio: Telegram

15 — Como implementar o canal Telegram?

R: Devem ser criadas TelegramNotificacao e TelegramFactory. TelegramNotificacao implementa Notificacao e seu método enviar() deve apresentar a mensagem com a indicação de Telegram. TelegramFactory herda de NotificacaoFactory e sobrescreve criarNotificacao() para retornar new TelegramNotificacao(). A regra de não modificar NotificacaoFactory é importante porque mostra que a fábrica abstrata já fornece a estrutura necessária para novos canais. O novo canal apenas se encaixa nessa estrutura por meio de herança, polimorfismo e implementação do contrato existente.

Atividade 12 — Comparação das soluções

16.1 — Qual solução possui maior acoplamento?

R: A solução inicial possui maior acoplamento. Nela, SistemaNotificacao conhece diretamente EmailNotificacao, SMSNotificacao e WhatsAppNotificacao e também é responsável por instanciá-las. Se uma dessas classes mudar ou se um novo tipo for acrescentado, SistemaNotificacao pode precisar ser alterada. Na solução com Factory Method, o cliente trabalha com abstrações e delega a criação às fábricas. Por isso, a dependência direta em relação aos produtos concretos fica concentrada nas fábricas concretas.

16.2 — Qual solução facilita a criação de novos tipos?

R: A solução com Factory Method facilita a criação de novos tipos. Para adicionar Push, por exemplo, basta criar PushNotificacao e PushFactory. Para Telegram, criamos TelegramNotificacao e TelegramFactory. As classes existentes permanecem intactas. Isso torna o sistema mais organizado para evolução, pois a criação de um novo produto não exige inserir novos blocos condicionais em uma classe central.

16.3 — Onde está localizada a responsabilidade de criação dos objetos?

R: A responsabilidade está nas fábricas concretas. EmailFactory conhece EmailNotificacao e sabe como criá-la; SMSFactory conhece SMSNotificacao, e assim por diante. A classe NotificacaoFactory fornece o ponto abstrato de criação, enquanto as subclasses definem a implementação. Dessa maneira, o código cliente não precisa conter todos os comandos de instanciação.

16.4 — Qual é a vantagem de trabalhar com Notificacao em vez de diretamente com EmailNotificacao?

R: A principal vantagem é poder trabalhar com uma abstração comum. Se o código dependesse diretamente de EmailNotificacao, ele estaria limitado ao canal de e-mail. Ao depender de Notificacao, o mesmo código pode receber EmailNotificacao, SMSNotificacao, WhatsAppNotificacao, PushNotificacao ou TelegramNotificacao, desde que implementem a interface. Isso permite polimorfismo, reduz acoplamento e facilita a extensão do sistema.

Atividade 13 — Factory Method × Strategy

17 — Qual representa comportamento e qual representa produto criado?

R: No Laboratório 01, DescontoStrategy representa comportamento. Cada estratégia define uma forma diferente de executar o cálculo ou aplicar uma regra de desconto. No Laboratório 02, Notificacao representa o produto criado. As classes concretas de notificação são objetos que podem ser produzidos pelas fábricas. Portanto, a diferença de foco é: Strategy organiza alternativas de comportamento, enquanto Factory Method organiza alternativas de criação de objetos.

Atividade 14 — Reflexão

Questão 1 — O que é o padrão Factory Method?

R: Factory Method é um padrão de projeto que define um método para criação de objetos e permite que subclasses determinem qual implementação concreta será criada. No laboratório, criarNotificacao() é o método responsável por essa criação. A ideia não é simplesmente colocar todos os comandos new em uma classe chamada Factory, mas separar a decisão e a responsabilidade de criação do código que utiliza o produto.

Questão 2 — Qual problema ele procura resolver?

R: Ele procura resolver principalmente problemas relacionados ao forte acoplamento na criação de objetos. Quando uma classe cria diretamente muitos produtos concretos, ela passa a depender de detalhes dessas classes. Além disso, cada novo tipo exige alterações no código existente. O Factory Method desloca a criação para uma estrutura especializada, permitindo que diferentes produtos sejam criados por meio de uma abstração comum.

Questão 3 — Qual é a responsabilidade de NotificacaoFactory?

R: NotificacaoFactory define a estrutura comum das fábricas. Ela declara o método abstrato criarNotificacao(), que representa o Factory Method, e implementa enviarMensagem(), que utiliza o produto retornado. Ela não determina diretamente se será criado e-mail, SMS ou WhatsApp. Sua função é fornecer o contrato e o fluxo comum que as fábricas concretas utilizarão.

Questão 4 — Qual é a responsabilidade de EmailFactory?

R: EmailFactory é responsável por decidir e executar a criação de uma EmailNotificacao. Ela sobrescreve criarNotificacao() e retorna new EmailNotificacao(). Assim, o conhecimento sobre a classe concreta fica localizado na fábrica correspondente, em vez de ficar espalhado pelo código cliente.

Questão 5 — Qual é a responsabilidade de EmailNotificacao?

R: EmailNotificacao é responsável por representar o produto concreto correspondente ao canal de e-mail. Ela implementa a interface Notificacao e fornece a implementação do método enviar(). Portanto, sua responsabilidade é executar a operação relacionada ao envio da notificação por esse canal, enquanto a responsabilidade de decidir quando e qual objeto criar pertence à fábrica.

Questão 6 — Por que SistemaNotificacao não precisa mais executar new EmailNotificacao()?

R: Porque a criação foi separada da utilização. A fábrica concreta é responsável por executar new EmailNotificacao(), enquanto o código que utiliza a notificação recebe uma referência do tipo Notificacao. Isso significa que SistemaNotificacao não precisa conhecer a implementação concreta nem controlar diretamente sua instanciação. Essa separação reduz o acoplamento e facilita a inclusão de novos produtos.

Questão 7 — Qual é a vantagem de retornar Notificacao em vez de EmailNotificacao?

R: Retornar Notificacao mantém a fábrica dependente da abstração, e não de uma implementação específica. Além disso, permite que diferentes produtos sejam utilizados por meio do mesmo tipo. Uma variável Notificacao pode referenciar EmailNotificacao, SMSNotificacao, WhatsAppNotificacao, PushNotificacao ou TelegramNotificacao. Assim, o código cliente pode chamar enviar() sem conhecer a classe concreta, aproveitando o polimorfismo.

Questão 8 — O que acontece quando adicionamos PushNotificacao?

R: Criamos uma nova implementação concreta, PushNotificacao, e uma nova fábrica, PushFactory. PushFactory herda de NotificacaoFactory e implementa criarNotificacao() para produzir PushNotificacao. As classes existentes não precisam ser modificadas. Esse comportamento demonstra a facilidade de extensão proporcionada pela estrutura do Factory Method.

Questão 9 — Por que podemos adicionar novos produtos sem modificar as fábricas existentes?

R: Porque cada novo produto pode possuir sua própria fábrica concreta. A nova fábrica apenas precisa seguir o contrato estabelecido por NotificacaoFactory. Por exemplo, TelegramFactory pode ser criada sem alterar EmailFactory, SMSFactory, WhatsAppFactory ou PushFactory. Cada fábrica conhece somente o produto que é sua responsabilidade criar. Essa separação evita que todas as fábricas dependam umas das outras.

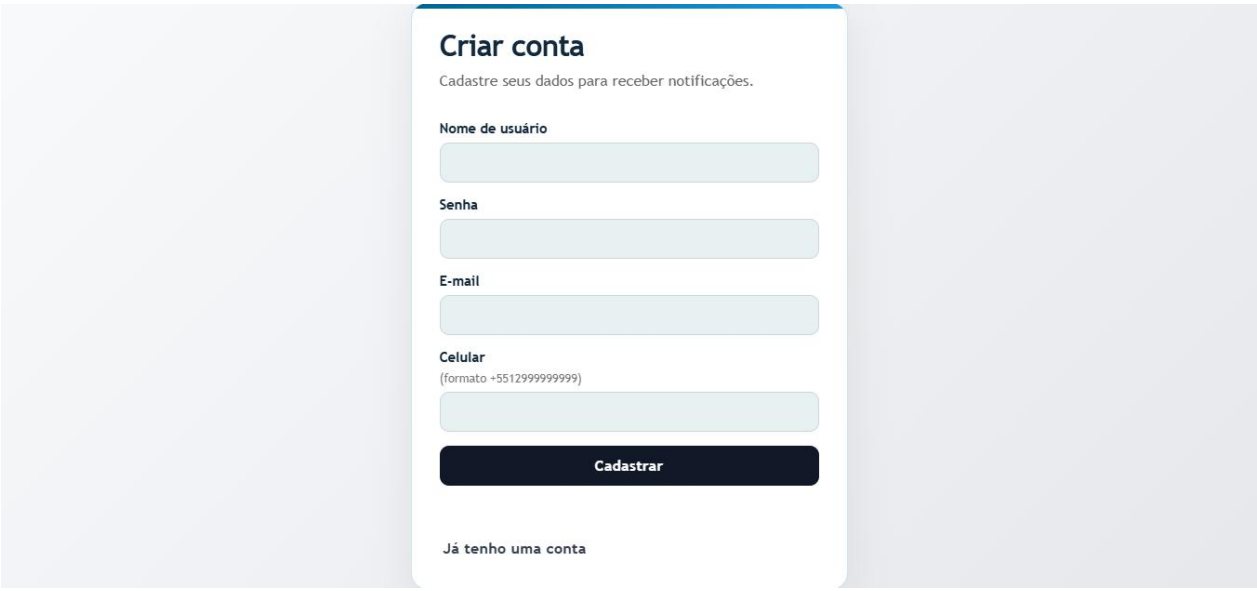
Questão 10 — Explique a diferença entre Strategy e Factory Method.

R: Strategy e Factory Method resolvem problemas diferentes. Strategy é utilizado para encapsular e permitir a troca de comportamentos ou algoritmos. No Laboratório 01, diferentes estratégias de desconto representam

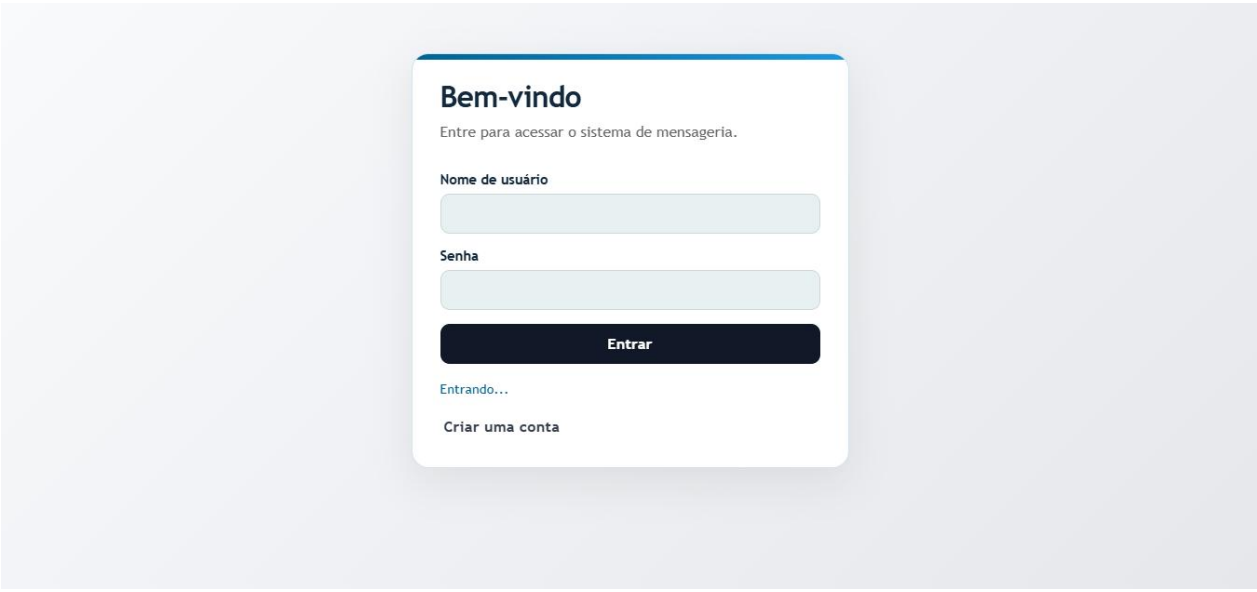
diferentes formas de executar uma regra. Factory Method, por outro lado, é utilizado para organizar a criação de objetos. No Laboratório 02, as fábricas determinam qual tipo de Notificacao será criado. Portanto, Strategy responde à pergunta 'qual comportamento devo utilizar?', enquanto Factory Method responde à pergunta 'qual objeto devo criar?'. Embora ambos utilizem abstrações, herança ou polimorfismo para permitir variações, o foco de cada padrão é diferente.

Print do funcionamento do projeto

Cadastro



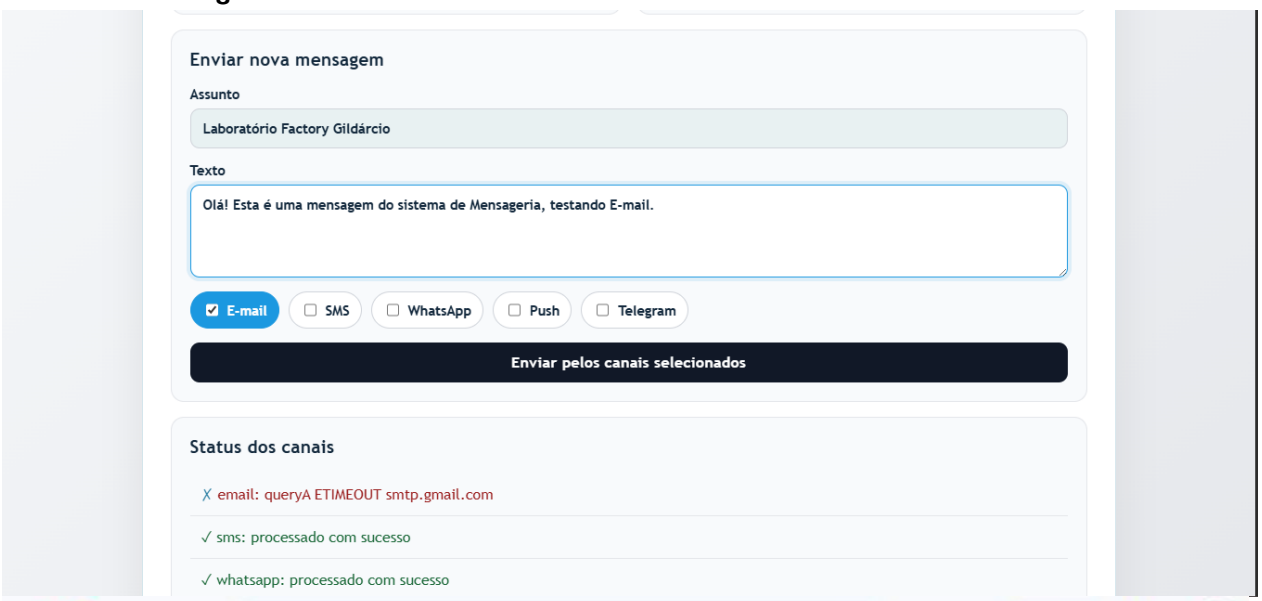
Login

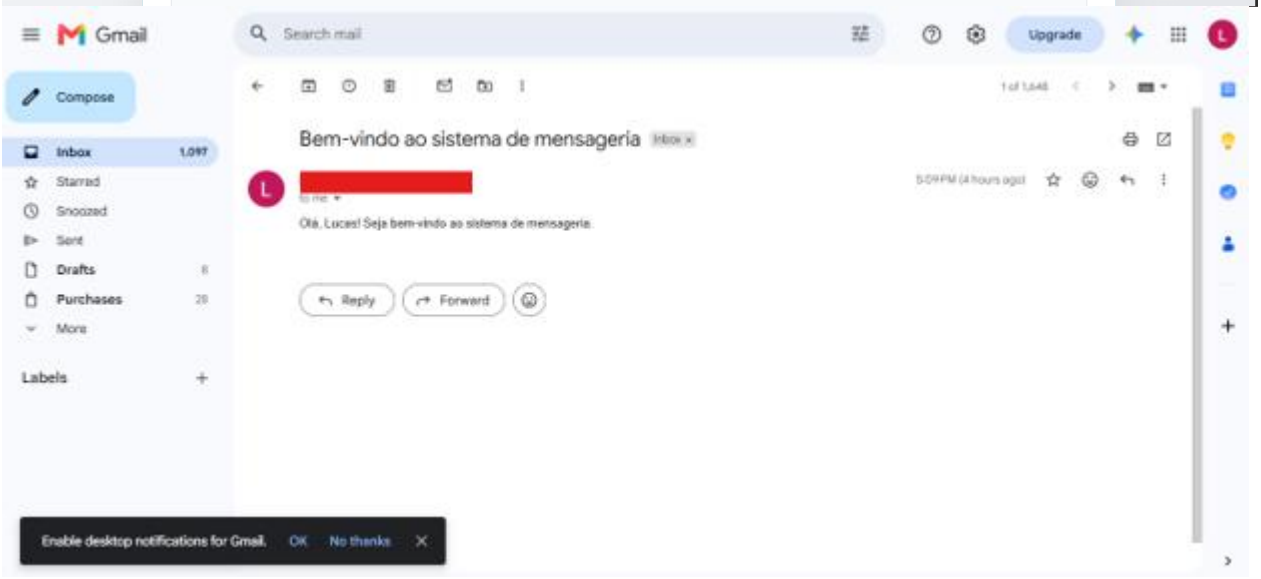


Sistema de Mensageria – Notificação

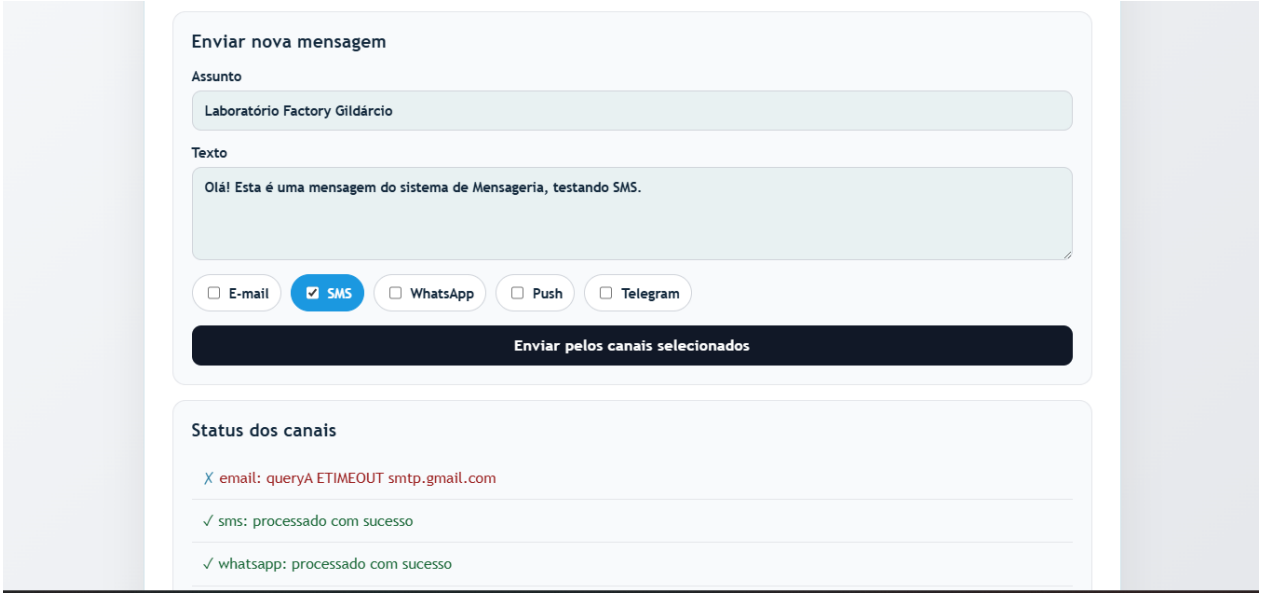


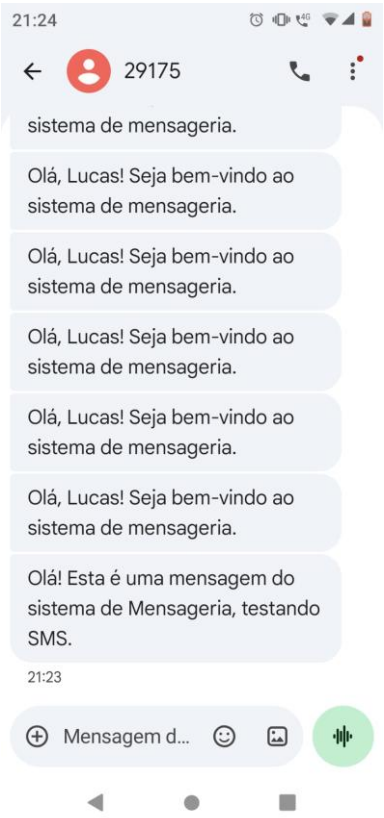
Sistema de Mensageria – E-mail





Sistema de Mensageria - SMS





Sistema de Mensageria – WhatsApp

